Universidad de San Carlos de Guatemala

Facultad de Ingeniería

Escuela de Ciencias y Sistemas

Introducción a la Programación y Computación 1

**Práctica 2: Visualizador de Algoritmos de Ordenamiento**

**Documentación Técnica**

Ibolya Orsolya David Cadenas

202530481

Guatemala, 29 de marzo de 2026

## Algoritmos

### Bubble Sort

Ordena comparando elementos adyacentes de manera consecutiva e intercambiándolos si están en desorden, esto lo hace tantas veces sea el tamaño del arreglo.

Pseudocódigo:

**Iterativo**

```
BubbleSort(arr)
    int temp = 0
    para(int i = 0; i<arr.length; i++){
        para(int j = 0; j<arr.length; j++)[
            if(arr[j]>arreglo[j+1]{
                temp = arreglo[j]
                arreglo[j] = arreglo[j+1]
                arreglo[j+1] = temp
            }
        }
    }
```

**Recursivo**

```
BubbleSort(arr, n):
    si n == 1:
        retornar
    para i desde 0 hasta n-2:
        si arr[i] > arr[i+1]
            intercambiar(arr[i], arr[i+1])
    BubbleSort(arr, n-1)
```

**Shell Sort**

Ordena a través de comparar elementos de un arreglo separados por intervalos de división de dos del tamaño del arreglo, moviendo los elementos distantes a sus posiciones correctas y posteriormente se disminuye esta división gradualmente hasta 1.

**Quick Sort**

Divide el arreglo utilizando un pivote y ordena recursivamente.

Pseudocódigo:

```
QuickSort(arr, low, high):
    si low < high
        pi = partition(arr, low, high)
        QuickSort(arr, low, pi-1)
        QuickSort(arr, pi+1, high)
```

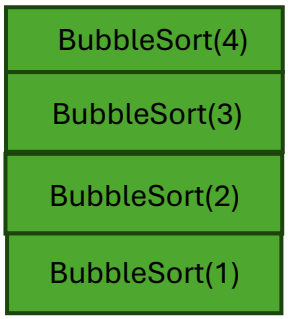
## Recursividad Aplicada

**Bubble Sort Recursivo**

**Caso Base:** Cuando la posición del arreglo a evaluar sea igual a 1.

**Caso Recursivo**: La comparación entre elementos consecutivos del arreglo de manera descendente para los índices del arreglo.

**Diagrama de Pila.** Para un arreglo de 4 elementos:

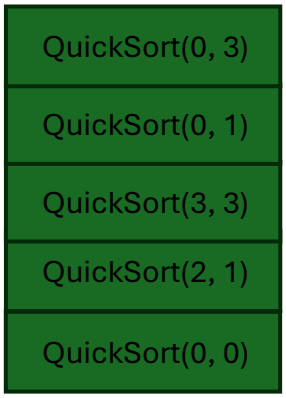


**Quick Sort**

**Caso base:** Al momento en que el índice izquierdo de donde se está ordenando sea mayor al derecho.

**Caso recursivo**: Analizando el pivote constantemente, se divide en dos el arreglo de forma recursiva.

**Diagrama de Pilas**. Considerando el arreglo [4, 3, 2, 1]




**Uso de Hilos y Comunicación con Swing**

Se implementó el uso de hilos para poder mostrar la ejecución de los diferentes algoritmos de ordenamientos junto con su visualización sin congelar la interfaz o interrumpir la ejecución principal del programa.

Para comunicar con Swing, se hizo uso del comando SwingUtilities.invokeLater(), el cual es un comando que permite ejecutar código en el hilo gráfico de Swing, permitiendo ejecutar tanto la parte gráfica como el código de manera ordenada.

**Biblioteca de Graficación**

Para el funcionamiento del proyecto se utilizó la siguiente biblioteca

Nombre: JFreeChart

Versión: 1.5.6

Uso: Representa gráficamente los datos del arreglo durante el proceso de ordenamiento.

**Reflexión Personal**

¿Qué diferencia encuentro entre la versión iterativa y la versión recursiva de Bubble Sort al ejecutarlas en modo comparación?

Al momento de ejecutarlas en el modo comparación, se muestran de la misma forma las comparaciones y los arreglos ordenados, lo que quiere decir que no hay diferencia en cuestión de la forma de ordenamiento del vector, sin embargo, si hay diferencia en cuestión del uso de memoria y el código realizado, por lo que implementar el método recursivo implica tomar en cuenta estos aspectos al momento de utilizarlo y verificar que es lo más adecuado para el tipo de caso.

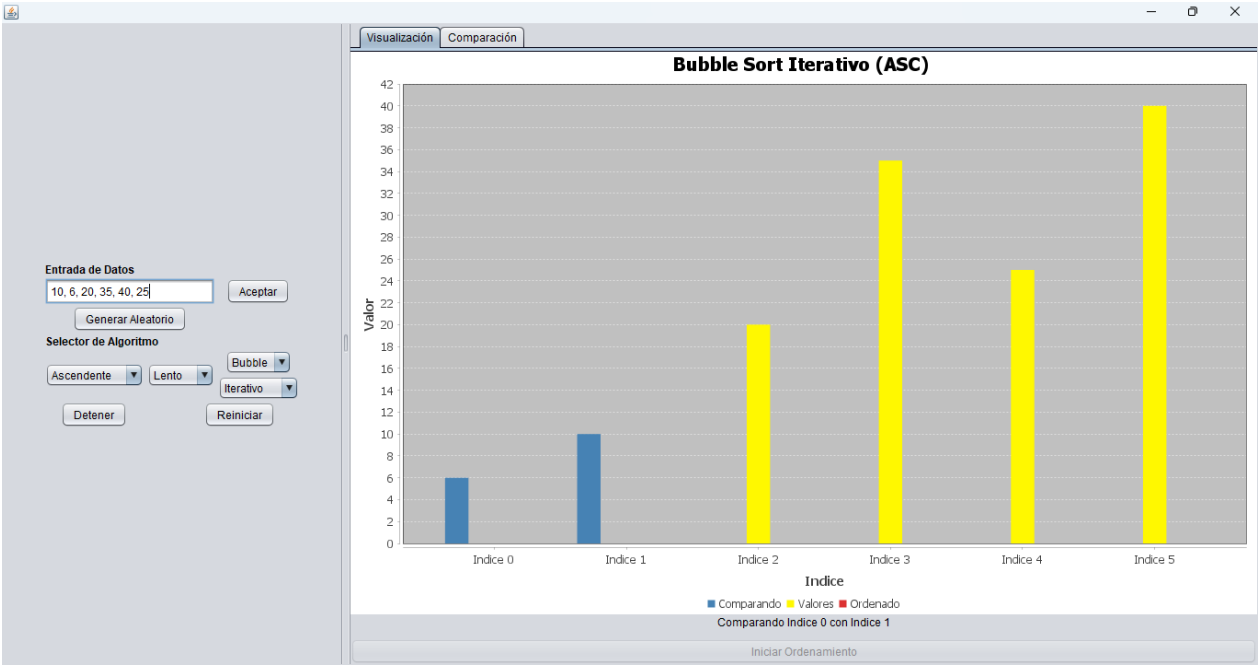
¿Por qué Quick Sort es naturalmente recursivo?

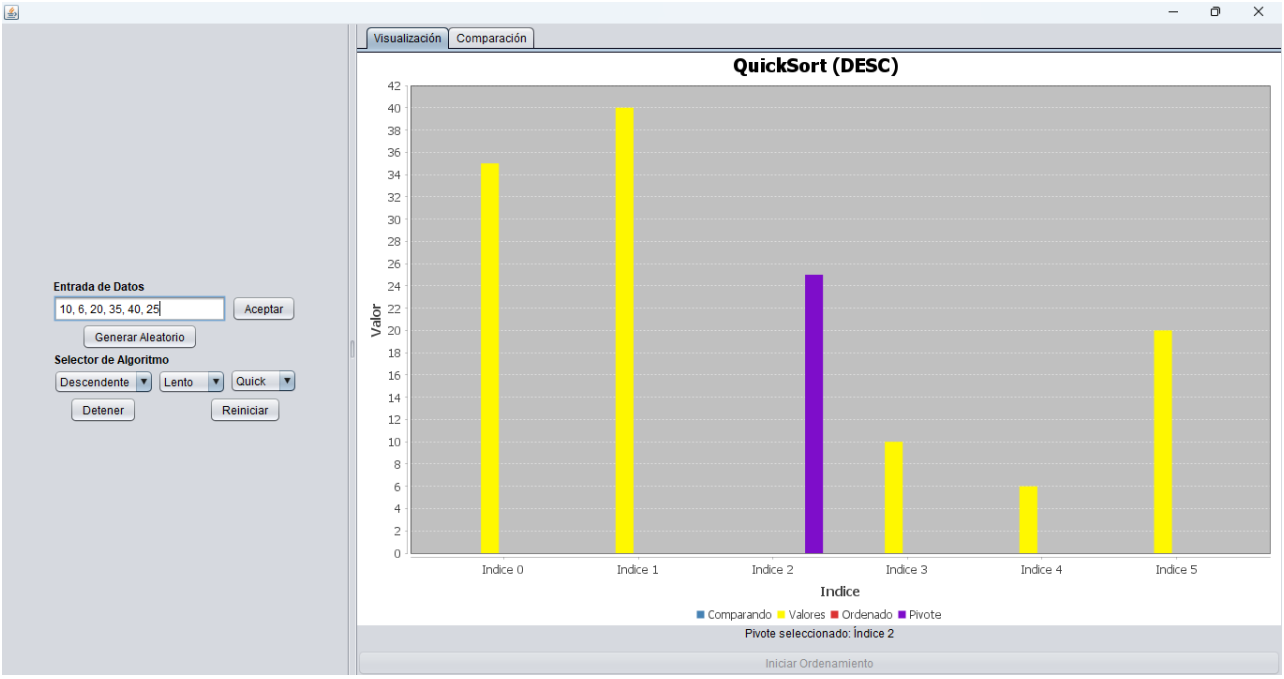
QuickSort es naturalmente recursivo porque divide constantemente el arreglo en mitades hasta que ya no haya más que dividir, por lo que la recursividad es su forma de poder representarle y ejecutarlo.

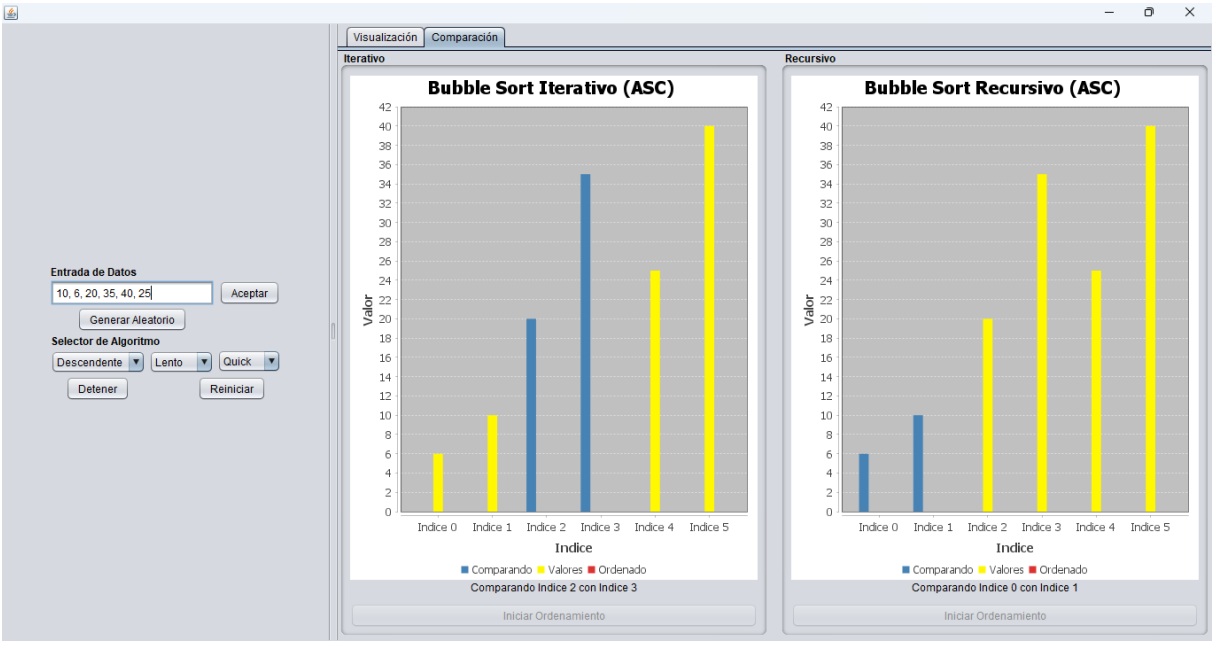
¿Qué ventajas o desventajas encontré en la recursividad frente a la iteración en este momento?

El mayor reto es identificar el caso que hará que termine con la recursividad, ya que se deben analizar varios aspectos del algoritmo e incluso su entera ejecución, una de sus ventajas es que el código se simplifica en gran parte.

# Anexos


Visualización
Comparación
Bubble Sort Iterativo (ASC)
Valor
Indice
Indice 0
Indice 1
Indice 2
Indice 3
Indice 4
Indice 5
Comparando
Valores
Ordenado
Comparando Indice 0 con Indice 1
Iniciar Ordenamiento
Entrada de Datos
10, 6, 20, 35, 40, 25
Aceptar
Generar Aleatorio
Selector de Algoritmo
Ascendente
Lento
Bubble
Iterativo
Detener
Reiniciar


Visualización
Comparación
QuickSort (DESC)
Valor
Indice
Indice 0
Indice 1
Indice 2
Indice 3
Indice 4
Indice 5
Comparando
Valores
Ordenado
Pivote
Pivote seleccionado: Índice 2
Iniciar Ordenamiento
Entrada de Datos
10, 6, 20, 35, 40, 25
Aceptar
Generar Aleatorio
Selector de Algoritmo
Descendente
Lento
Quick
Detener
Reiniciar


Visualización
Comparación
Entrada de Datos
10, 6, 20, 35, 40, 25
Aceptar
Generar Aleatorio
Selector de Algoritmo
Descendente
Lento
Quick
Detener
Reiniciar
Iterativo
Bubble Sort Iterativo (ASC)
Valor
Indice
Indice 0
Indice 1
Indice 2
Indice 3
Indice 4
Indice 5
Comparando
Valores
Ordenado
Comparando Indice 2 con Indice 3
Iniciar Ordenamiento
Recursivo
Bubble Sort Recursivo (ASC)
Valor
Indice
Indice 0
Indice 1
Indice 2
Indice 3
Indice 4
Indice 5
Comparando
Valores
Ordenado
Comparando Indice 0 con Indice 1
Iniciar Ordenamiento